

MAG

Documentação Técnica do Front-End

Showcase • fe-traditional-sale

ARQUITETURA • APIs • FLUXOS • IMPLEMENTAÇÃO

DIREÇÃO VISUAL

Base institucional #003366, tipografia sem serifa, cartões brancos, neutros frios e estrutura preparada para substituição da cor primária por parceiro.

Agosto de 2026

Guia visual aplicado

Referência de identidade para esta documentação

Primária #003366	Texto #0F172A	Fundo #F4F6F8	Borda #E2E8F0
NOTA	A cor #003366 é a referência padrão. Em parceiros com identidade própria, apenas a cor primária deve mudar; neutros, tipografia, espaçamento e estrutura permanecem estáveis.		

Conteúdo técnico

Arquitetura, contratos, páginas e fluxos do front-end

- 01 Introdução
- 02 Arquitetura de Alto Nível
- 03 Fluxo de Dados & Estado
- 04 Contratos de API
- 05 Páginas Front-End
- 06 Access.jsx em detalhe
- 07 Demais páginas
- 08 Implementação técnica
- 09 Diagramas
- 10 Como acessar
- 11 Checklist do Showcase

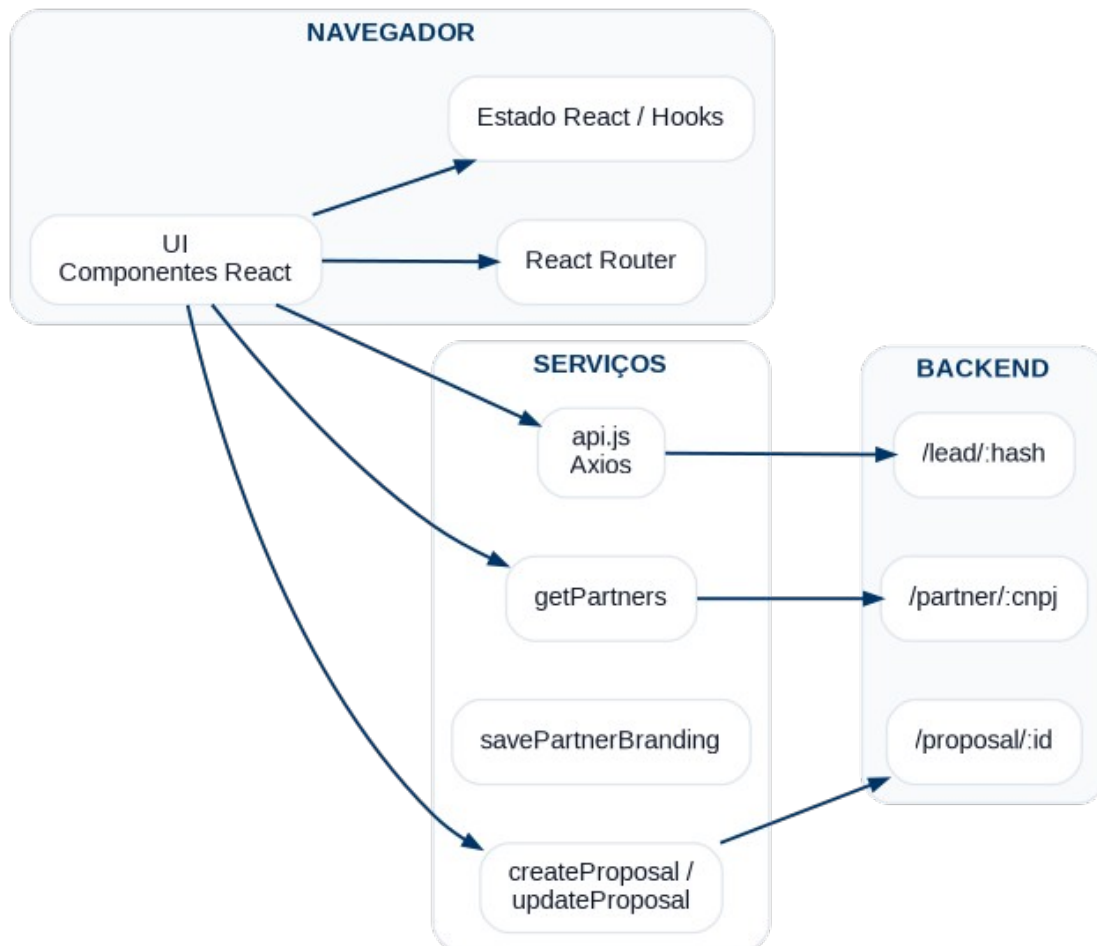
1. Introdução

Este documento fornece uma visão técnica abrangente da aplicação **MAG** front-end localizada no repositório [fe-traditional-sale](#). Destina-se a um showcase da equipe e cobre:

- Arquitetura de alto nível e fluxo de dados
- Contratos de API do backend (endpoints, formatos de requisição/resposta)
- Descrição detalhada de cada componente de página em [src/pages/](#)
- Passo a passo aprofundado da tela **Access** ([src/pages/Access.jsx](#))
- Detalhes de implementação (gerenciamento de estado, serviços, rotas, sessionStorage)
- Diagramas Mermaid e tabelas de API para clareza visual

O documento está escrito em **Markdown** e pode ser convertido para **PDF** para a apresentação.

2. Arquitetura de Alto Nível



A aplicação é uma **SPA React** construída com Vite. A comunicação com o backend ocorre via Axios (src/services/api.js). O roteamento é gerenciado por react-router-dom.

3. Fluxo de Dados & Gerenciamento de Estado

Origem	Destino	Meio	Descrição
UI (campos de formulário)	Estado local do componente (useState)	Estado React	Captura entradas do usuário (CNPJ, hash, IDs de corretor, etc.)
Componente (Access.jsx)	sessionStorage	Armazenamento do navegador	Persiste IDs do corretor entre páginas (@Mag:brokerExternalId, @Mag:brokerProducerId, @Mag:brokerSusep)
Componente	Cabeçalhos padrão do Axios	Memória	Define o header <code>cnpj</code> para todas as chamadas subsequentes
Serviço <code>partnerService</code>	Backend <code>/partner/:cnpj/</code>	HTTP GET	Busca dados de branding do parceiro
Serviço <code>proposalService</code>	Backend <code>/proposal/</code>	HTTP POST/PUT	Cria e atualiza objetos de proposta
Componente (Access.jsx)	<code>navigate</code> (React Router)	Alteração de URL	Redireciona para <code>/products</code> ou <code>/proposalflow?id=...</code>
Componente	sessionStorage (lead draft)	Armazenamento do navegador	Guarda temporariamente o rascunho do lead quando não há oferta

4. Contratos de API do Backend

4.1 Busca de Lead (`GET /api/sales-cap/v1/lead/:hash`)

JSON

```
GET /api/sales-cap/v1/lead/:hash
Resposta (200):
{
  "data": {
    "generalInfo": {
      "leadId": "string",
      "partnerVendorId": "string",
      "businessPartnerDocument": "string",
      "offerId": "string",
      "offerCode": "string",
```

```
"totalContribution": "number",
"monthTerm": "number",
...
},
"commercialDetails": {
  "vendors": [{
    "mainVendor": true,
    "externalId": "string",
    "producerId": "string",
    "susepId": "string"
  }]
},
"policyHolders": [{
  "personDetails": {"document": "string", "name": "string", ...},
  "emails": [{"email": "string", "main": true}],
  "telephones": [{"nationalDestinationCode": "string", "number": "string", "main": true}],
  "address": {...}
}]
}
}
```

4.2 Dados do Parceiro (`GET /api/partner/:cnpj`)

JSON

```
GET /api/partner/:cnpj
Resposta (200):
{
  "legalName": "string",
  "branding": {"logoUrl": "string", "colors": {...}},
  ...
}
```

4.3 Criação de Proposta (`POST /api/proposal`)

JSON

```
POST /api/proposal
Body:
{
  "productId": "string",
  "offerCode": "string",
  "quantity": 1,
  "totalValue": number,
  "partnerCnpj": "string",
  "partnerName": "string",
  ...
}
Resposta (201):
{ "id": "uuid", "status": "draft", ... }
```

4.4 Atualização de Proposta (`PUT /api/proposal/:id`)

JSON

```
PUT /api/proposal/:id
```

```

Body (parcial):
{
  "client_type": "juridica|fisica",
  "document_number": "string",
  "form_data": { ... }
}
Resposta (200):
{ "id": "uuid", "updated": true }

```

5. Visão Geral das Páginas Front-End (`src/pages/`)

Componente	Rota	Propósito	Principais Props / Parâmetros
Access.jsx	/ (home)	Ponto de entrada – localizar lead por hash ou inserir dados do parceiro manualmente	Estado: hashLead, cnpj, externalId, producerId, susep
Products.jsx	/products	Exibe catálogo de produtos para o parceiro selecionado	Lê @Mag:brokerExternalId do sessionStorage
ProposalFlow.jsx	/proposalflow	Assistente passo a passo da criação da proposta	Query id na URL para identificar a proposta
Extrato.jsx	/extrato	Exibe extrato de transações da proposta	Recebe proposalId via estado de roteamento
Envio.jsx	/envio	Upload de documentos/arquivos da proposta	Recebe proposalId da etapa anterior
ProductsSalesAssistant.jsx	/assistant	Assistente de vendas com IA	Integração com serviço externo de IA
Backup/ (pasta)	—	Scripts/utilitários de backup (não UI)	—
ProposalPDF/ (pasta)	—	Geração de PDF da proposta (lado servidor)	—

6. Detalhamento – `Access.jsx`

6.1 Referência do Arquivo

[Access.jsx](file:///c:/Users/T-GAMER/Documents/Empresa/MAG/fe-traditional-sale/src/pages/Access.jsx)

6.2 Visão Geral do Componente

JSX

```
export default function Access() { ... }
```

O componente renderiza a **tela inicial** onde o usuário pode:

1. **Buscar um lead** usando o **hash** (consulta automática de parceiro e corretor).
2. **Inserir manualmente** as credenciais do parceiro (CNPJ + campos opcionais do corretor).
3. **Continuar** para o catálogo de produtos ou fluxo de proposta.

6.3 Variáveis de Estado

Variável	Tipo	Finalidade
hashLead	string	Hash do lead fornecido pelo usuário
cnnpj	string	CNPJ do parceiro (numérico, formatado)
externalId	string	ID externo do corretor
producerId	string	ID do produtor do corretor
susep	string	Registro SUSEP do corretor
isLoading	boolean	Exibe spinner durante chamadas assíncronas
leadMessage	string	Mensagens de validação/erro da busca de lead
isLookingUpLead	useRef<boolean>	Impede buscas concorrentes de lead

6.4 Regras de Negócio (validação)

JS

```
const showBrokerFields = cnnpj.length >= 14;  
const isValidForm = cnnpj.length >= 14 && externalId !== "" && susep !== "";
```

- Campos do corretor são exibidos somente quando o **CNPJ é válido** (≥ 14 dígitos).
- O botão **Continuar** fica habilitado quando o formulário está válido **ou** um hash foi informado.

6.5 Funções Principais

`handleLeadLookup`

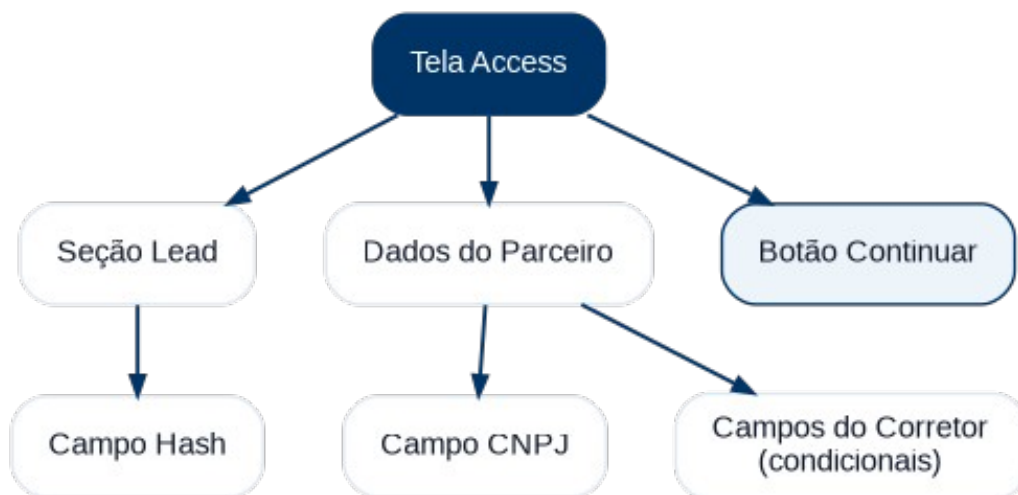
- **Entrada:** Trim em `hashLead`; aborta se vazio ou se já há busca em andamento.
- **Chamada API:** GET `/api/sales-cap/v1/lead/:hash` via `api.get`.

- **Processamento:** Extrai CNPJ do parceiro, IDs do corretor e dados do segurado.
- **Side-effects:**
 - Armazena IDs do corretor em `sessionStorage`.
 - Executa `getPartners` → `savePartnerBranding` para cache de branding.
 - Monta `leadDraft` (tipo de cliente, documento, dados do formulário).
 - Caso o lead **não** contenha produto/oferta, grava `leadDraft` e redireciona para `/products`.
 - Caso contrário, cria proposta (`createProposal`) e a atualiza com dados do cliente (`updateProposal`).
 - Por fim, navega para `/proposalflow?id=....`
- **Tratamento de Erro:** Exibe mensagem amigável ao usuário.

`handleAccess``

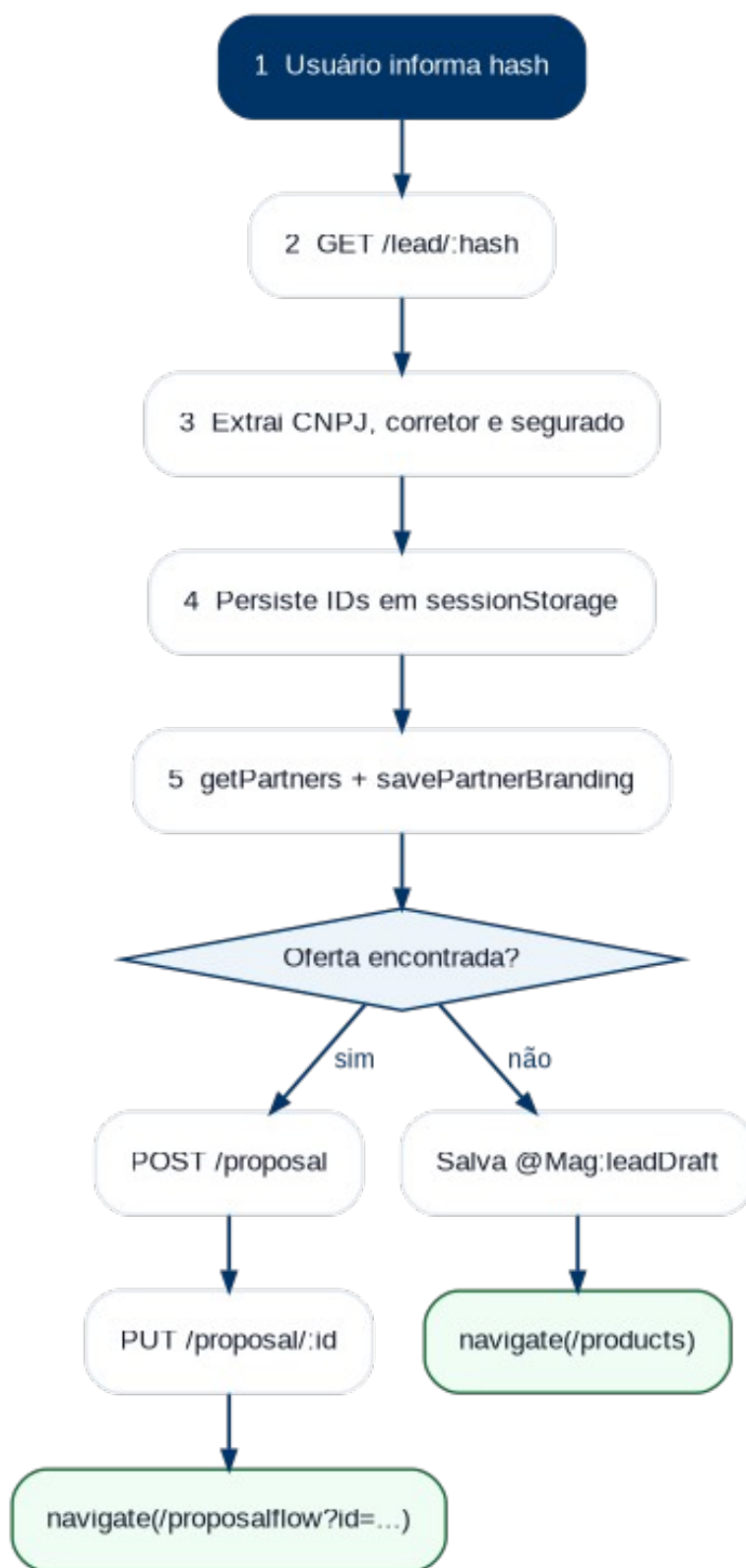
- **Caminho 1 – Hash informado:** Invoca `handleLeadLookup`.
- **Caminho 2 – Entrada manual:**
 1. Valida formulário (`isFormValid`).
 2. Busca branding do parceiro via `getPartners(cnpj)`.
 3. Persiste IDs do corretor em `sessionStorage`.
 4. Define header padrão `cnpj` no Axios para chamadas subsequentes.
 5. Navega para `/products`.
- **Controle de Loading:** Gerenciado por `setIsLoading`.

6.6 Estrutura da UI (simplificada)



- **Seção Lead:** Campo de hash com disparo `onBlur` para busca.
- **Seção Manual:** Campo CNPJ + campos opcionais do corretor (ID externo, ID produtor, SUSEP).
- **Botão:** Desabilitado até validação; mostra `*“Consultando dados...”*` enquanto `isLoading`.

6.7 Fluxo de Interação



O diagrama ilustra as duas ramificações possíveis após a busca de lead.

7. Outras Páginas – Resumo rápido

7.1 `Products.jsx`

- **Rota:** `/products`
- **Objetivo:** Listar produtos de seguro disponíveis para o parceiro selecionado.
- **Lógica principal:** Lê `@Mag:brokerExternalId` do `sessionStorage`; obtém lista via `api.get('/products')`.
- **Interação:** Clique em um produto cria um **rascunho de proposta** e encaminha para `ProposalFlow`.

7.2 `ProposalFlow.jsx`

- **Rota:** `/proposalflow`
- **Objetivo:** Assistente passo a passo (dados do cliente → escolha do produto → resumo → geração de PDF).
- **Estado:** Gerido por `ProposalContext` (React Context) para persistência entre etapas.
- **API:** Cria proposta na primeira etapa (`createProposal`) e a atualiza com dados do cliente.

7.3 `Extrato.jsx`

- **Rota:** `/extrato`
- **Objetivo:** Exibir extrato de transações de uma proposta finalizada.
- **Fonte de dados:** Busca proposta pelo ID (estado do roteador) e apresenta tabela de parcelas.

7.4 `Envio.jsx`

- **Rota:** `/envio`
- **Objetivo:** Upload de documentos (PDF, imagens) vinculados à proposta.
- **Implementação:** Usa `FormData` e `api.post('/proposal/:id/attachments')`.

7.5 `ProductsSalesAssistant.jsx`

- **Rota:** `/assistant`
- **Objetivo:** Assistente de IA que sugere produtos com base no perfil do cliente.
- **Serviço externo:** Chama `POST /ai/suggest` e exibe sugestões em interface de chat.

8. Detalhes Técnicos de Implementação

8.1 Serviços (src/services)

Serviço	Funções Exportadas	Descrição
api.js	api (instância Axios)	Cliente HTTP central, define <code>baseUrl</code> , interceptors de autenticação e tratamento de erros
partnerService.js	getPartners(cnpj)	GET <code>/partner/:cnpj</code> – devolve branding e metadados do parceiro
partnerBranding.js	savePartnerBranding(cnpj, data)	Persiste branding em <code>localStorage</code> sob a chave <code>@Mag:partnerBranding:\${cnpj}</code>
proposalService.js	createProposal(payload), updateProposal(id, payload)	Operações CRUD de propostas

8.2 Rotas (src/App.jsx)

```
JSX

import { BrowserRouter, Routes, Route } from 'react-router-dom';

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Access />} />
    <Route path="/products" element={<Products />} />
    <Route path="/proposalflow" element={<ProposalFlow />} />
    <Route path="/extrato" element={<Extrato />} />
    <Route path="/envio" element={<Envio />} />
    <Route path="/assistant" element={<ProductsSalesAssistant />} />
  </Routes>
</BrowserRouter>
```

Todas as páginas são **protegidas** por uma simples validação em `Access.jsx` que garante CNPJ/corretor válidos antes da navegação.

8.3 Estilização

- Estilos são escopados via CSS Modules (`*.module.css`).
- O design segue a paleta da empresa definida em `src/pages/Access.module.css` – usa variáveis CSS para cores primárias/secundárias e suporte a modo escuro via `prefers-color-scheme`.

8.4 Session & Local Storage

- Chaves `sessionStorage` com prefixo `@Mag`: armazenam dados temporários de sessão (IDs do corretor, rascunho de lead).
- `localStorage` é usado por `partnerBranding.js` para cache de branding entre sessões.

9. Resumo dos Diagramas

- **Diagrama de Componentes** (seção 2) visualiza Navegador ↔ Serviços ↔ Backend.
- **Fluxo da Tela Access** (seção 6.7) mostra ações do usuário → API → navegação.
- **Diagrama de Sequência** (seção 6.7) detalha a lógica de ramificação da busca de lead.

10. Como Acessar a Tela Access

1. Inicie o servidor de desenvolvimento (`npm run dev`).
2. Abra o navegador em <http://localhost:5173/> (URL padrão do Vite).
3. A página inicial renderiza o componente **Access** descrito acima.
4. Você pode:
 - Colar um **hash de lead** e perder o foco do campo (a busca ocorre automaticamente), ou
 - Preencher o **CNPJ** e, se necessário, os campos do corretor, e clicar em **Continuar**.
5. Se a busca for bem-sucedida, o fluxo redireciona automaticamente para **Products** ou **ProposalFlow**.

11. Checklist para o Showcase

- ✓ Diagrama de arquitetura incluído (Mermaid).
- ✓ Tabelas de contrato de API adicionadas.
- ✓ Descrição completa de cada página.
- ✓ Passo a passo detalhado da tela Access.jsx com estado, validações e fluxo.
- ✓ Resumos das demais páginas.
- ✓ Documento salvo como um único arquivo Markdown pronto para conversão em PDF.

FIM

Documentação técnica consolidada para showcase do front-end MAG.

BRANDING DINÂMICO

O documento adota #003366 como cor institucional de referência. A mesma estrutura suporta substituição da cor primária por parceiro, sem alteração dos neutros, tipografia ou organização visual.